VeriTix

Sistema de Gestión de Eventos y Tickets Digitales

Fase 2: Ejecución del Proyecto y Pruebas

Erick Quispe · Juan Rangel

DAM/DAW

Proyecto Fin de Ciclo

2026

Los documentos, elementos gráficos, vídeos, transparencias y otros recursos didácticos incluidos en este contenido pueden contener imprecisiones técnicas o errores tipográficos. Periódicamente se realizan cambios en el contenido. Fomento Ocupacional FOC SL puede realizar en cualquier momento, sin previo aviso, mejoras y/o cambios en el contenido.

Es responsabilidad del usuario el cumplimiento de todas las leyes de derechos de autor aplicables. Ningún elemento de este contenido, ni parte de este contenido puede ser reproducida, almacenada o introducida en un sistema de recuperación, ni transmitida de ninguna forma ni por ningún medio, sin la previa autorización por escrito de Fomento Ocupacional FOC SL.

Este contenido está protegido por la ley de propiedad intelectual e industrial. Pertenecen a Fomento Ocupacional FOC SL los derechos de autor y los demás derechos de propiedad intelectual e industrial sobre este contenido.

Sin perjuicio de los casos en que la ley aplicable prohíbe la exclusión de la responsabilidad por daños, Fomento Ocupacional FOC SL no se responsabiliza en ningún caso de daños indirectos que se deriven del uso de este contenido.

© 2026 Fomento Ocupacional FOC SL todos los derechos reservados.

Índice general

1 Estudio inicial previo a la realización del proyecto **5**
1.1 Clasificación de empresas del sector por organización y servicio . . . 5
1.1.1 Organizadores de eventos . . . 5
1.1.2 Plataformas de ticketing . . . 5
1.1.3 Operadores de acceso y validación . . . 5
1.2 Ejemplo de estructura organizativa del sector . . . 5
1.3 Necesidades que cubre VeriTix . . . 6
1.4 Descripción del proyecto . . . 6
1.5 Justificación del tipo de solución . . . 6
1.6 Características principales del sistema . . . 7

2 Identificación de necesidades y diseño del proyecto **8**
2.1 Estudio inicial y planificación del proyecto . . . 8
2.1.1 Identificar las fases del proyecto y su contenido . . . 8
2.1.2 Especificar los objetivos del proyecto . . . 8
2.1.3 Especificar recursos hardware y software . . . 9
2.1.4 Especificar recursos materiales y personales . . . 9
2.1.5 Asociación entre fases y recursos (materiales y humanos) . . . 9
2.2 Aspectos fiscales y laborales . . . 10
2.2.1 Obligaciones de protección de datos (RGPD/LOPD-GDD) . . . 10
2.2.2 Aspectos fiscales . . . 10
2.2.3 Prevención de riesgos laborales . . . 10
2.3 Viabilidad económica . . . 10
2.3.1 Presupuesto económico . . . 10
2.3.2 Financiación necesaria . . . 11
2.3.3 Ayudas y subvenciones . . . 11
2.4 Modelo de solución . . . 11
2.4.1 Modelado de la solución . . . 11
Arquitectura general del sistema . . . 11
Módulos backend (estado verificable) . . . 11
API real del sistema . . . 12
Modelo de datos y constraints . . . 13
Diagrama entidad-relación (resumen) . . . 15
Seguridad y control de integridad . . . 16
2.4.2 Puntos de control para validación del proyecto . . . 16

3 Ejecución del proyecto y pruebas **17**
3.1 Riesgos de ejecución del proyecto . . . 17
3.1.1 Identificación de riesgos . . . 17
3.1.2 Plan de prevención y mitigación . . . 17

3.2 Documentación de ejecución 17
3.2.1 Necesidades legales 18
3.2.2 Ejecución del proyecto (estado verificable) 18
3.2.3 Contenerización y despliegue operativo 18
3.3 Incidencias 19
3.3.1 Protocolo de resolución 19
Recopilación de información 19
Diagnóstico y solución 19
Registro 20

4 Pruebas y soporte **21**
4.1 Documento de pruebas a realizar 21
4.1.1 Trazabilidad con contratos API y modelo de datos 21
4.1.2 Pruebas unitarias (backend) 21
4.1.3 Pruebas de integración y e2e (backend) 21
4.1.4 Pruebas de concurrencia/carga 21
4.1.5 Pruebas de seguridad funcional 21
4.1.6 Pruebas de acceso y usabilidad (frontend) 22
4.2 Registro de pruebas (evidencia documental) 22
4.3 Evaluación de cumplimiento de objetivos 22
4.4 Soporte y mantenimiento 23

1. Estudio inicial previo a la realización del proyecto

1.1. Clasificación de empresas del sector por organización y servicio

El sector de gestión de eventos y ticketing digital puede agruparse en tres tipos de organizaciones:

1.1.1. Organizadores de eventos

Promotoras, instituciones y asociaciones que necesitan publicar eventos, controlar aforo y gestionar ventas desde una plataforma única.

1.1.2. Plataformas de ticketing

Empresas que comercializan entradas de terceros y operan con alto volumen transaccional, especialmente en aperturas de venta con picos de concurrencia.

1.1.3. Operadores de acceso y validación

Equipos y empresas que gestionan el acceso físico al recinto y requieren validación rápida, trazable y resistente a intentos de reutilización de entradas.

1.2. Ejemplo de estructura organizativa del sector

Una ticketera digital de tamaño medio suele organizarse en la siguiente matriz funcional:

Área funcional	Responsabilidad principal
Dirección	Estrategia y priorización de negocio.
Operaciones	Relación con organizadores y operación de eventos.
Tecnología	Backend, frontend, base de datos, seguridad y despliegue.
Comercial/marketing	Captación de organizadores y promoción.
Atención al cliente	Incidencias de compra y acceso.
Validación en campo	Control de acceso durante el evento.

1.3. Necesidades que cubre VeriTix

VeriTix aborda necesidades reales del sector con alcance verificable en el repositorio:

Necesidad del sector	Implementación verificable en VeriTix
Centralización operativa	Gestión de eventos, catálogos y usuarios desde una API unificada.
Control de stock y sobreventa	Compra con transacciones atómicas en PostgreSQL/Prisma.
Trazabilidad del acceso	Tickets con hash único y registro de validación (`validatedAt`, `validatedBy`).
Automatización del flujo de cobro	Integración con Stripe Checkout + webhooks.
Notificación transaccional	Envío de emails de verificación y confirmación mediante Resend y BullMQ.

1.4. Descripción del proyecto

VeriTix es un monorepo con dos aplicaciones desacopladas:

Componente	Tecnología
Backend	NestJS 11 + TypeScript + Prisma 7 + PostgreSQL + Redis.
Frontend	Nuxt 4 + Vue 3 + Nuxt UI + Tailwind 4.

La autenticación se basa en JWT (access token de corta vida y refresh token en cookie HTTP-only con rotación). El dominio contempla roles `BUYER`, `CREATOR`, `VALIDATOR` y `ADMIN`.

1.5. Justificación del tipo de solución

Se selecciona arquitectura web desacoplada (API REST + cliente web SSR/SPA) por:

Criterio	Justificación técnica
Escalabilidad operativa	Separación de backend y frontend para evolucionar cada capa.
Mantenibilidad	Módulos de dominio en NestJS y tipado fuerte extremo a extremo.
Portabilidad	La API puede reutilizarse por otros clientes (ej. app móvil).
Accesibilidad	Acceso multiplataforma sin instalación de software cliente nativo.

1.6. Características principales del sistema

Capacidad	Implementación actual
Autenticación y roles por servidor	Validación de permisos por endpoint.
Compra transaccional con Stripe	Confirmación por webhook y generación de tickets.
Validación por hash único	Trazabilidad de uso y auditoría de acceso.
Catálogos de dominio	Artistas, recintos, géneros y formatos.
Caché con Redis	Optimización de consultas frecuentes.

2. Identificación de necesidades y diseño del proyecto

2.1. Estudio inicial y planificación del proyecto

2.1.1. Identificar las fases del proyecto y su contenido

La planificación original se definió en ocho fases. A continuación se refleja el grado de cumplimiento según el estado real del repositorio (abril de 2026).

Fase	Contenido planificado	Estado actual
1. Análisis y diseño	Requisitos, modelo de datos, contratos API, wireframes	Completada
2. Autenticación y usuarios	Registro/login/refresh/roles	Completada (backend + frontend base)
3. Gestión de eventos y catálogos	CRUD de eventos, recintos, artistas, géneros y formatos	Completada
4. Venta de tickets	Órdenes, Stripe Checkout, webhooks, generación de tickets	Completada en backend; integración frontend pendiente
5. Validación de tickets	Endpoint de validación y trazabilidad de acceso	Backend completado; UI de scanner pendiente
6. Panel administrativo	Gestión y métricas para administración	Completada (área admin en frontend)
7. Testing	Pruebas unitarias/integración/e2e/concurre	Implementadas en backend; no ejecutadas en esta auditoría
8. Despliegue y documentación	Operación productiva y manuales finales	Parcial

2.1.2. Especificar los objetivos del proyecto

- Centralizar la gestión de eventos y tipos de ticket.
- Automatizar venta, cobro y emisión digital de tickets.
- Evitar sobreventa con consistencia transaccional.
- Asegurar validación de acceso trazable por ticket.
- Mantener un modelo de permisos por rol en backend y frontend.

2.1.3. Especificar recursos hardware y software

Hardware de referencia

- Equipos de desarrollo para ejecutar backend, frontend y servicios auxiliares.
- Dispositivos móviles para validación de UI responsive y pruebas de lectura QR.
- Servidor de despliegue (cuando se opere en producción).

Stack software real del proyecto

Capa / dominio	Tecnologías y mecanismos
Backend	NestJS 11, Prisma 7, PostgreSQL, Redis, BullMQ.
Frontend	Nuxt 4, Vue 3, Nuxt UI, Tailwind 4.
Auth	JWT + refresh token HTTP-only con rotación.
Pagos	Stripe Checkout + webhook (`/api/v1/webhooks/stripe`).
Notificaciones	Resend (emails) + colas BullMQ.
Testing	Jest, Supertest y suites de concurrencia.

2.1.4. Especificar recursos materiales y personales

Recursos materiales

- Infraestructura local de desarrollo y repositorio GitHub.
- Servicios externos de pago y correo para entorno real (Stripe, Resend).

Recursos personales

Rol	Responsabilidad
Equipo de desarrollo	Backend, frontend y base de datos.
Tutor	Seguimiento técnico y validación académica.

2.1.5. Asociación entre fases y recursos (materiales y humanos)

Fase	Recursos materiales	Recursos humanos
Análisis/diseño	Herramientas de modelado y documentación	Equipo técnico del proyecto
Desarrollo backend/frontend	IDE, control de versiones, servicios locales	Equipo técnico del proyecto
Testing	Suites automáticas + dispositivos de prueba	Equipo técnico + tutor

Fase	Recursos materiales	Recursos humanos
Revisiones por hito	Documentación de avance y evidencias	Tutor académico
Despliegue / cierre	Servidor, reverse proxy, certificados TLS	Equipo técnico + tutor

2.2. Aspectos fiscales y laborales

2.2.1. Obligaciones de protección de datos (RGPD/LOPD-GDD)

El sistema trata datos personales de usuarios registrados (identidad y contacto), por lo que debe garantizar información al usuario, base legal del tratamiento, medidas de seguridad y gestión de derechos ARSOPL.

2.2.2. Aspectos fiscales

Para explotación comercial, las operaciones de venta deben integrarse con obligaciones de facturación e impuestos aplicables según jurisdicción.

2.2.3. Prevención de riesgos laborales

En el contexto de desarrollo software, los riesgos principales son ergonómicos y visuales derivados del trabajo prolongado con pantallas, mitigables con pausas y condiciones de puesto adecuadas.

2.3. Viabilidad económica

2.3.1. Presupuesto económico

Concepto	Coste estimado
Desarrollo realizado por el equipo académico	0€ (coste imputado no monetario)
Dominio y hosting básico anual	Bajo (dependiente de proveedor)
Licencias software base	0€ (stack open source)

2.3.2. Financiación necesaria

La fase académica requiere financiación reducida. Para fase comercial, la estructura de costes debe recalcularse según volumen transaccional, SLAs y soporte.

2.3.3. Ayudas y subvenciones

Como opción futura, el proyecto podría optar a programas de emprendimiento o digitalización, condicionado a formalización empresarial y plan financiero específico.

2.4. Modelo de solución

2.4.1. Modelado de la solución

Arquitectura general del sistema

VeriTix adopta arquitectura cliente-servidor desacoplada en monorepo. El frontend consume exclusivamente la API REST del backend.

Aspecto	Backend	Frontend
Framework	NestJS 11	Nuxt 4 / Vue 3
Puerto local habitual	3001	3000
Responsabilidad	Lógica de negocio, seguridad, persistencia	UI, navegación y consumo de API
Persistencia	Prisma 7 + PostgreSQL	Sin acceso directo a BD

Módulos backend (estado verificable)

La implementación backend se organiza en dominios funcionales claramente delimitados:

Dominio funcional	Módulos
Identidad y acceso	`auth`, `users`
Catálogo y eventos	`events` (incluye `ticket-types` y `event-artists`)
Transacción y emisión	`orders`, `tickets`, `webhooks`
Infraestructura de dominio	`venues`, `artists`, `genres`, `concert-formats`
Soporte operativo	`notifications`, `queues`, `cache`

API real del sistema

El backend aplica prefijo global configurable mediante `API_PREFIX` y, en ausencia de valor en entorno, utiliza `api/v1` (`backend/src/main.ts`). Por tanto, las rutas públicas/protegidas se consumen como `/api/v1/...`.

Los contratos siguientes se extrajeron de controladores y DTOs del repositorio (`backend/src/modules/*`). Se rediseñan en tablas compactas para mantener legibilidad en PDF.

Autenticación

Endpoint	Seguridad	Resumen funcional
POST /api/v1/auth/register	Pública	Alta usuario y devuelve mensaje de verificación
POST /api/v1/auth/login	Pública	Recibe credenciales y devuelve token + cookie refresh
POST /api/v1/auth/refresh	Cookie refresh válida	Emite nuevo access token y rota cookie
POST /api/v1/auth/logout	JWT + cookie refresh	Revoca refresh token y limpia cookie (204)

Eventos y catálogo

Endpoint	Seguridad	Resumen funcional
GET /api/v1/events	Pública	Lista eventos con paginación y filtros
POST /api/v1/events	JWT + rol ADMIN/CREATOR	Crea evento
POST /api/v1/events/:id/publish	JWT + rol ADMIN/CREATOR	Publica evento existente
POST /api/v1/events/:eventId/ticket-types	JWT + rol ADMIN/CREATOR	Crea tipo de ticket para un evento

Órdenes, tickets y pagos

Endpoint	Seguridad	Resumen funcional
POST /api/v1/orders	JWT usuario autenticado	Crea orden y retorna checkoutUrl cuando aplica
GET /api/v1/orders/my	JWT usuario autenticado	Lista órdenes del comprador
GET /api/v1/tickets/mine	JWT usuario autenticado	Lista tickets del comprador

Endpoint	Seguridad	Resumen funcional
POST /api/v1/tickets/validate	JWT + rol ADMIN/VALIDATOR	Valida ticket por hash y registra trazabilidad
POST /api/v1/webhooks/stripe	Firma stripe-signature válida	Procesa evento de pago/reembolso

Cobertura de flujo (backend vs frontend).

Cap	Estado verificable
Back	Contratos de auth, eventos, órdenes, tickets y webhook Stripe implementados, con suites de pruebas en `backend/test/` y `backend/src/**/*.spec.ts`.
Fron	Cobertura para autenticación, catálogo público de eventos y área administrativa; los flujos de compra de órdenes, consumo de tickets del comprador y scanner/validación final en UI permanecen parciales o pendientes.

Modelo de datos y constraints

El esquema Prisma (`backend/prisma/schema.prisma`) se organiza en dos bloques:

Bloque	Entidades
Núcleo transaccional	User, RefreshToken, Event, TicketType, Order, OrderItem, Ticket, Payment.
Catálogos y soporte	Venue, Artist, Genre, ConcertFormat, EventArtist.

Relaciones principales verificables

Relación	Cardinalidad
User (creator) con Event	1:N
Event con TicketType, Order y Ticket	1:N
Order con OrderItem, Payment y Ticket	1:N
TicketType con OrderItem y Ticket	1:N
User (buyer) con Order y Ticket	1:N
User (validator) con Ticket validado	1:N (opcional)
Event con Artist mediante EventArtist	N:M
Event con Genre y Artist con Genre	N:M

Constraints e índices relevantes (extracto)

Tipo de restricción	Detalle
Unicidad	`users.email`, `users.phone`, `tickets.hash`, `venues.slug`, `artists.slug`, `genres.name/slug`, `concert_formats.name/slug`, `event_artists(event_id, artist_id)`.
Integridad referencial (FK)	Presente en las relaciones críticas (`event_id`, `buyer_id`, `order_id`, etc.).
Cascadas de borrado	`RefreshToken→User`, `EventArtist→Event`, `TicketType→Event`, `OrderItem→Order`, `Ticket→Event`.
Índices de consulta	`events(status, eventDate)`, `orders(buyerId, createdAt)`, `orders(eventId, status)`, `tickets(buyerId, status)`, `tickets(eventId, status)`.
Enums de dominio	`Role`, `EventStatus`, `OrderStatus`, `TicketStatus`, `PaymentStatus`.

Limitaciones actuales del esquema (a considerar)

Limitación	Implicación
Ausencia de CHECK constraints	No se observan restricciones como `availableQuantity >= 0`, `totalQuantity >= availableQuantity` o `saleStartDate <= saleEndDate`.
`currency` como String	Requiere validación estricta de ISO.
FKs sin `onDelete` explícito en algunos casos	Conviene revisar políticas de borrado para producción.

Diagrama entidad-relación (resumen)

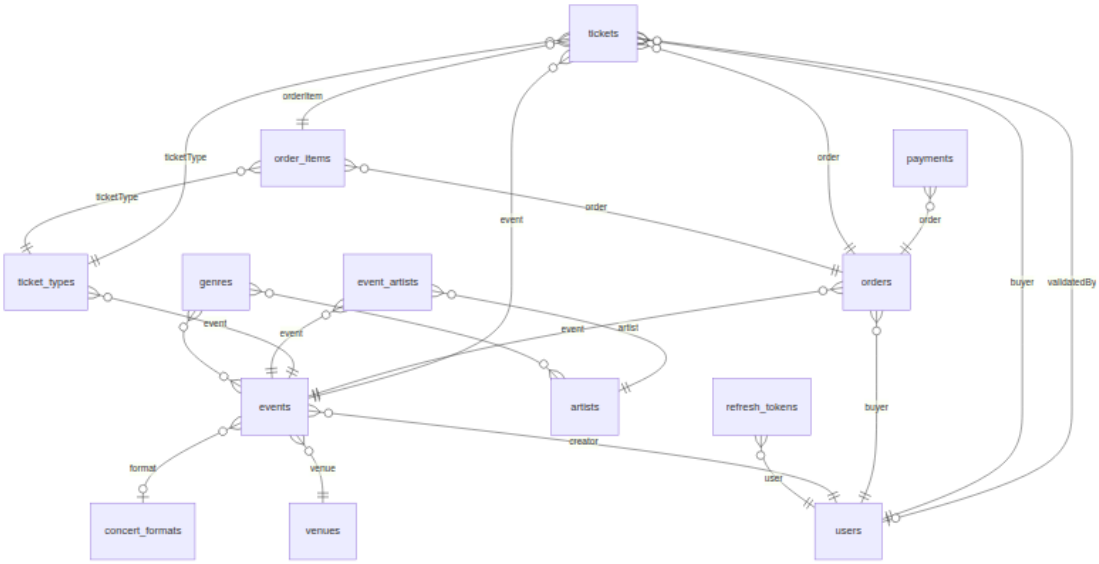

Figura 2.1. *Vista general del diagrama ER*

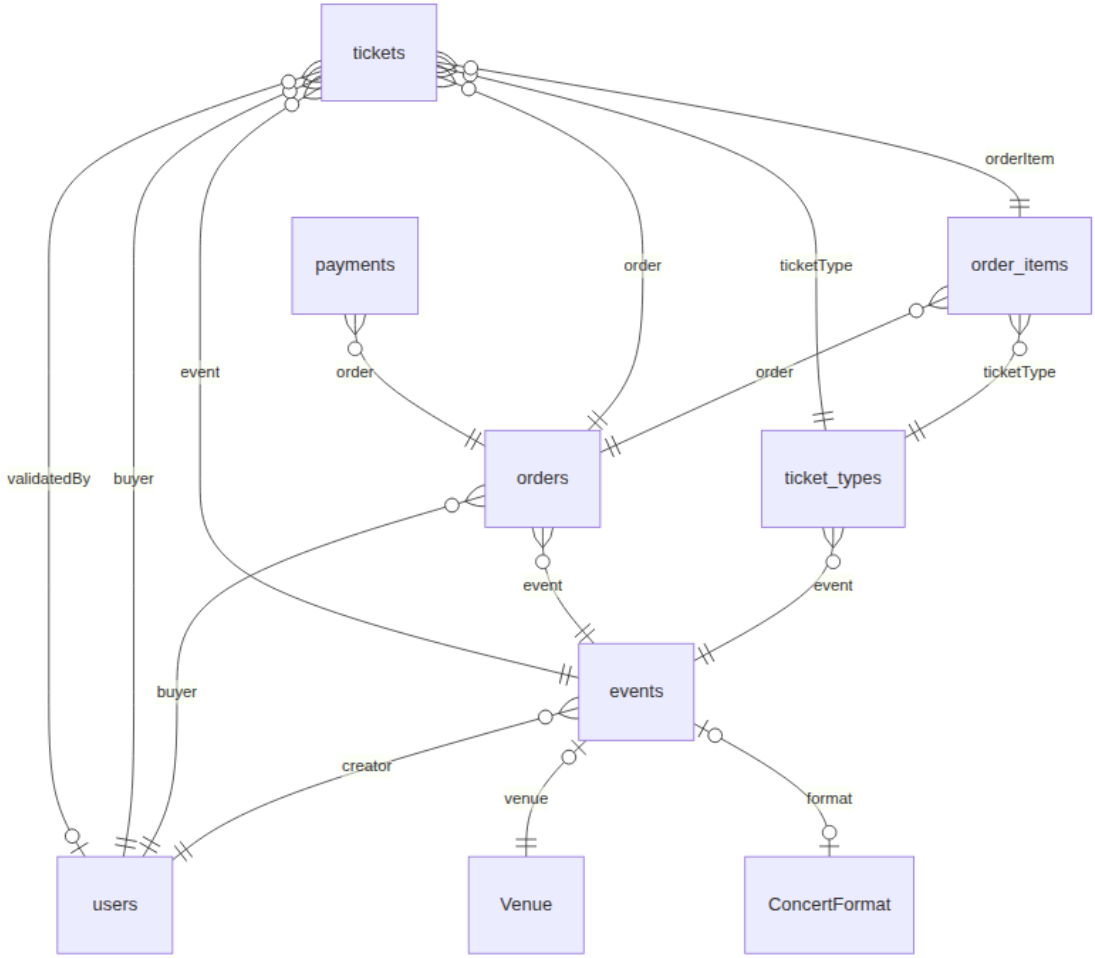

Figura 2.2. *Dominio transaccional del sistema*

Seguridad y control de integridad

Mecanismo	Función
JWT + refresh token rotativo	Autenticación y renovación segura de sesión.
Joi en `env.validation.ts`	Validación de variables de entorno críticas.
`prisma.$transaction()`	Aislamiento de operaciones críticas de compra.
Hash SHA-256 por ticket	Identificación y validación del ticket vía `qrPayload`.

2.4.2. Puntos de control para validación del proyecto

Punto de control	Verificación
Integridad y unicidad del ticket	Validación de uso único en acceso.
Prevención de sobreventa	Consistencia bajo concurrencia.
Control de acceso por rol	Protección de endpoints sensibles.
Auditoría de validación	Registro de `validatedAt` y `validatedBy`.
Confirmación de pago	Webhook y emisión de notificaciones.

3. Ejecución del proyecto y pruebas

3.1. Riesgos de ejecución del proyecto

3.1.1. Identificación de riesgos

ID	Riesgo	Probabilidad	Impacto
R01	Desalineación entre funcionalidades backend y frontend	Media	Alto
R02	Errores en flujos de pago/webhook con efectos en órdenes y tickets	Media	Alto
R03	Defectos de seguridad en autenticación o manejo de secrets	Baja	Muy alto
R04	Degradación bajo concurrencia en compra de tickets	Media	Alto
R05	Desactualización documental respecto al código real	Alta	Medio

3.1.2. Plan de prevención y mitigación

Riesgo	Mitigación
R01	Mantener matriz de estado funcional por capa (backend/frontend) y actualizarla en cada hito.
R02	Pruebas de integración y e2e específicas de órdenes, tickets y webhooks.
R03	Validación estricta de entorno, revisión de permisos y rotación de credenciales.
R04	Mantener y ampliar suites de concurrencia (`test:concurrency`) y stress.
R05	Auditorías documentales periódicas con trazabilidad a archivos de código.

3.2. Documentación de ejecución

3.2.1. Necesidades legales

En el contexto académico no se requieren licencias de actividad. Para explotación comercial sí aplica cumplimiento en protección de datos, comercio electrónico y obligaciones fiscales.

3.2.2. Ejecución del proyecto (estado verificable)

Ficheros de configuración relevantes:

- `backend/.env.example`: variables necesarias del backend (`DATABASE_URL`, secrets JWT, Stripe, Resend, Redis, CORS, etc.).
- `backend/src/config/env.validation.ts`: validación Joi de variables críticas en arranque.
- `backend/prisma/schema.prisma`: modelo de datos y restricciones.
- `backend/docker-compose.yml` y `frontend/docker-compose.yml`: configuración de contenedores por paquete. (No existe un `docker-compose.yml` único en la raíz).

3.2.3. Contenerización y despliegue operativo

La solución está containerizada y preparada para operarse en una VPS con servicios aislados. El backend levanta PostgreSQL, Redis y la API en contenedores separados; el frontend usa un `Dockerfile` multi-stage con runtime no-root.

Frontend (Dockerfile multi-stage)

```
FROM oven/bun:1-alpine AS base
WORKDIR /app

FROM base AS deps
COPY package.json bun.lock ./
RUN bun install --frozen-lockfile

FROM deps AS builder
COPY . .
RUN bun run build

FROM oven/bun:1-alpine AS production
WORKDIR /app
ENV NODE_ENV=production
ENV NITRO_HOST=0.0.0.0
ENV NITRO_PORT=3000
USER nuxt
EXPOSE 3000
CMD ["bun", ".output/server/index.mjs"]
```

Backend (orquestación de servicios)

```yaml
services:
    postgres:
        image: postgres:18-alpine
        restart: unless-stopped
        ports:
            - '127.0.0.1:5432:5432'

    redis:
        image: redis:8-alpine
        restart: unless-stopped
        ports:
            - '127.0.0.1:6379:6379'

    backend:
        build:
            dockerfile: Dockerfile.dev
        ports:
            - '3001:3001'
```

Aspectos técnicos confirmados:

- API con prefijo global `/api/v1`.
- Documentación Swagger activa en `/docs`.
- Webhook de Stripe en `/api/v1/webhooks/stripe` con `rawBody` habilitado para firma.
- Seguridad con `helmet`, `cookie-parser`, validación global y guards.

3.3. Incidencias

3.3.1. Protocolo de resolución

Recopilación de información

Para cada incidencia se debe registrar: contexto funcional, pasos de reproducción, rol afectado, hora, entorno (local/staging/prod), trazas relevantes y severidad.

Diagnóstico y solución

1. Clasificar incidencia (funcional, seguridad, rendimiento, datos, UX).
2. Reproducir en entorno controlado.
3. Aplicar corrección mínima verificable.
4. Ejecutar pruebas de regresión del módulo impactado.

Registro

Las incidencias deben documentarse en GitHub Issues/PR con: descripción, evidencia, raíz del problema, cambio aplicado, pruebas asociadas y estado de cierre.

4. Pruebas y soporte

4.1. Documento de pruebas a realizar

La estrategia de pruebas se estructura por niveles y distingue entre evidencia existente en repositorio, contratos del backend, esquema Prisma y cobertura aún pendiente de integración funcional en frontend.

4.1.1. Trazabilidad con contratos API y modelo de datos

La identificación de endpoints reales, constraints de datos y resumen entidad-relación se documenta en `docs/latex/src/02-identificacion-necesidades.md` (subsecciones: *API real del sistema*, *Modelo de datos y constraints* y *Diagrama entidad-relación* (*resumen*)). Esta referencia permite alinear criterios de prueba con contratos efectivamente implementados en backend.

4.1.2. Pruebas unitarias (backend)

Cobertura de servicios y controladores en módulos críticos (auth, events, orders, tickets, webhooks, catálogos), con pruebas localizadas en `backend/src/**/**.spec.ts`.

4.1.3. Pruebas de integración y e2e (backend)

Suites en `backend/test/` para auth, eventos, órdenes, tickets, catálogos y concurrencia.

4.1.4. Pruebas de concurrencia/carga

Scripts de concurrencia y stress declarados en `backend/package.json` (`test:concurrency`, `test:load`, `test:stress`, `test:soak`).

4.1.5. Pruebas de seguridad funcional

Validación de autenticación JWT, permisos por rol y comportamiento de validación de tickets.

4.1.6. Pruebas de acceso y usabilidad (frontend)

Frontend con tests unitarios/nuxt/e2e vía Vitest; no obstante, el flujo completo de compra y scanner QR no está integrado en UI en el estado actual.

4.2. Registro de pruebas (evidencia documental)

ID	Evidencia (resumen)	Estado
T01	Unitaria backend: backend/src/…/tickets.generator.spec.ts	Disponible
T02	Unitaria backend: backend/src/…/tickets.service.spec.ts	Disponible
T03	Integración/e2e auth: backend/test/auth.e2e-spec.ts	Disponible
T04	Integración/e2e events: backend/test/events.e2e-spec.ts	Disponible
T05	Integración/e2e orders: backend/test/orders.e2e-spec.ts	Disponible
T06	Integración/e2e tickets: backend/test/tickets.e2e-spec.ts	Disponible
T07	Concurrencia: backend/test/concurrency/orders.concurrency.spec.ts	Disponible
T08	Seguridad config: backend/src/config/env.validation.ts	Disponible
T09	Frontend tests: scripts Vitest en frontend/package.json	Disponible
T10	Compra E2E UI: sin rutas frontend de órdenes/tickets	Pendiente
T11	Ticket PDF UI: flujo frontend no implementado	Pendiente
T12	Scanner QR UI: pantalla/flujo final no integrado	Pendiente

4.3. Evaluación de cumplimiento de objetivos

Requisito	Verificación (resumen)	Estado
OBJ-01 Gestión de eventos	CRUD y administración disponibles	Cumplido
OBJ-02 Seguridad de tickets	Hash único + validación backend	Cumplido (backend)
OBJ-03 Anti-sobreventa	Lógica transaccional + pruebas concurrencia	Cumplido
OBJ-04 Roles y permisos	Guards y restricciones por endpoint	Cumplido
OBJ-05 Compra UX completa	Flujo completo desde frontend	Parcial
OBJ-06 Entrega ticket al comprador	Visualización/descarga en frontend	Parcial
OBJ-07 Escaneo QR móvil	UI de scanner integrada	Pendiente

Requisito	Verificación (resumen)	Estado
OBJ-08 Trazabilidad validaciones	validatedAt / validatedBy en tickets	Cumplido
OBJ-09 Prueba bajo carga	Scripts/suites declarados	Disponible (sin ejecución en auditoría)
OBJ-10 Consistencia doc-código	Documentación alineada a estado real	Mejorada

4.4. Soporte y mantenimiento

- Mantener trazabilidad entre issue, PR y prueba asociada.
- Priorizar correcciones en flujos críticos: autenticación, órdenes, tickets y webhooks.
- Versionar cambios documentales junto con cambios funcionales para evitar deriva de contenido.